



Assignment 2 – Interference Engine

Idris Anver: 103491762

Harry Ward 103641884

Institutional Affiliation(s)

Course Number: COS30019 Introduction to AI

1. Introduction

This report describes the implementation of an inference engine for propositional logic, based on a software-based Truth Table (TT), Backward chaining (BC) and Forward chaining (FC) algorithms. This Inference engine takes arguments from a Horn-form Knowledge base (KB) and a query (q) which is a proposition symbol and determines whether q can be entailed from KB.

2. Instructions

To run the program, execute the command prompt, navigate to the program directory, type 'python InferenceEngine.py', and within the command line Arguments you are able to choose between:

- Forward Chaining
- Truth Table
- Backward Chaining
- Resolution based

The format should be: python InferenceEngine.py 'type of algorithm' 'name of test file'

E.g: python InferenceEngine.py FC test.txt

3. Implementation

In exploring logical inference, the Implementation of Truth Table (TT), Backward Chaining (BC), and Forward Chaining (FC) algorithms stand as foundational pillars. TT exhaustively evaluates all possible combinations, while BC and FC navigate through rules, either backward or forward, to deduce conclusions. These algorithms form the bedrock of logical reasoning systems, here we aim to explain how we implement all our algorithms. As seen in our 'main' (InferenceEngine.py), each algorithm is set in its own individual file. For insight on how each method works, please check each algorithms file for comments on the process.

TT: A truth table is a logical tool used in mathematics and computer science to systematically analyse the truth values of logical expressions. It displays all possible combinations of inputs for the variables in an expression, along with the corresponding output. Each row of the table represents a unique combination of truth values for the variables, and the final column shows the resulting truth value of the expression under consideration. Truth tables are valuable for understanding the behaviour of logical

operations, verifying the validity of logical expressions, and determining if they are tautologies, contradictions, or contingencies. They are fundamental in logic design and theorem proving.

FC: Forward chaining is a reasoning technique employed in artificial intelligence and expert systems to infer conclusions from a set of initial facts and rules. Unlike backward chaining, which starts with a goal and works backward to determine if it's satisfied, forward chaining begins with known facts and uses rules to derive new information iteratively. It involves repeatedly applying rules to infer new facts until no further conclusions can be drawn. This bottom-up approach is often used in systems where the end goals are not predefined or when there's a need to explore a large space of possible conclusions.

BC: Backward Chaining is a reasoning strategy used in artificial intelligence and expert systems to infer the causes or goals behind observed facts. Unlike forward chaining, which starts with known facts to derive conclusions, backward chaining begins with a goal to determine what needs to be true for that goal to be achieved. It works by recursively applying rules and reasoning backward from the goal through antecedents until reaching known facts or base cases. This process effectively traces back from a desired outcome to the conditions required for its fulfilment, making it valuable in problem-solving, diagnosis, and decision-making applications.

RB: Resolution-based theorem proving is a cornerstone of automated reasoning in propositional logic, playing a crucial role in artificial intelligence, formal verification, and computational logic. The resolution method is based on the principle of refutation. The process begins with a set of clauses representing the knowledge base (KB) and a query. The query is negated and added to the KB. The objective is to derive an empty clause, which signifies a contradiction. If a contradiction is found, it means that the original query logically follows from the KB. Otherwise, if no contradiction is found, the query is not entailed by the KB. Before applying the resolution rule, it is essential to convert all logical sentences into a standard format known as clausal form. Clausal form is a conjunction of disjunctions of literals. A literal is either an atomic proposition or its negation. The resolution rule is the heart of this method. It operates on two clauses that contain complementary literals—one literal and its negation. The resolution process is iterative. Starting with the initial set of clauses, the algorithm repeatedly applies the resolution rule to pairs of clauses to generate new clauses. Each new clause is added to the set of clauses if it is not already present. The process continues until one of two conditions is met, an empty clause is derived, indicating a contradiction and thus proving that the original

query is entailed by the KB or No new clauses can be generated, indicating that the query is not entailed by the KB. Resolution-based theorem proving is a systematic and rigorous method for determining logical entailment. By converting propositions into clausal form and iteratively applying the resolution rule, this method effectively checks for contradictions to confirm or refute logical queries. Its efficiency and robustness make it a foundational tool in various domains of computer science and artificial intelligence.

4. Testing

In exploring logical propositions, testing the truth tables serves as a fundamental method. This systematic examination reveals the veracity of logical statements by exhaustively assessing every possible combination of truth values for their components, providing a rigorous validation process.

TestTruthTable Class

1. **test_simple_kb:**
 - Tests a simple knowledge base (KB) where 'p' is true.
 - Expected result: 'YES: 1'.
2. **test_empty_kb:**
 - Tests an empty KB with no symbols or sentences.
 - Expected result: 'NO'.
3. **test_implications_only_kb:**
 - Tests a KB with implications that do not lead to 'b' being true.
 - Expected result: 'NO'.
4. **test_edge_case_kb:**
 - Tests an edge case KB with mixed true and false sentences.
 - Expected result: 'NO'.
5. **test_multiple_true_sentences:**
 - Tests a KB with multiple true sentences.
 - Expected result: 'YES: 2'.
6. **test_single_false_sentence:**
 - Tests a KB with a single false sentence.
 - Expected result: 'NO'.
7. **test_mixed_sentences:**
 - Tests a KB with mixed true and false sentences.
 - Expected result: 'NO'.

TestForwardChaining Class

1. **test_implications_only_kb:**
 - Tests a KB with implications where 'a' does not lead to 'b'.
 - Expected result: 'NO'.
2. **test_contradictory_kb:**
 - Tests a contradictory KB where 'f' cannot be proven because 'g' is not in the KB.
 - Expected result: 'NO'.
3. **test_chain_of_implications:**
 - Tests a chain of implications where 'a' leads to 'b', 'b' leads to 'c', and 'c' leads to 'd'.
 - Expected result: 'YES: a, b, c, d'.
4. **test_multiple_implications:**
 - Tests multiple implications where 'a' implies both 'b' and 'c'.
 - Expected result: 'YES: a, c'.

TestBackwardChaining Class

1. **test_implications_only_kb:**
 - Tests a KB with implications where 'a' does not lead to 'b'.
 - Expected result: 'NO'.
2. **test_contradictory_kb:**
 - Tests a contradictory KB where 'f' cannot be proven because 'g' is not in the KB.
 - Expected result: 'NO'.
3. **test_simple_fact_kb:**
 - Tests a KB with a single fact 'a'.
 - Expected result: 'YES: a'.
4. **test_chain_of_implications:**
 - Tests a chain of implications where 'a' leads to 'b', 'b' leads to 'c', and 'c' leads to 'd'.
 - Expected result: 'YES: a, b, c, d'.
5. **test_multiple_implications:**
 - Tests multiple implications where 'a' implies both 'b' and 'c'.
 - Expected result: 'YES: a, c'.

TestResolutionBased Class

1. **test_fact_only_kb:**
 - Tests a KB with only facts and no contradictions.
 - Expected result: `False`.
2. **test_implications_only_kb:**
 - Tests a KB with only implications and no contradictions.
 - Expected result: `False`.
3. **test_edge_case_kb:**
 - Tests an edge case KB where a contradiction is expected.
 - Expected result: `True`.
4. **test_contradictory_kb:**
 - Tests a contradictory KB where no contradiction is found.
 - Expected result: `False`.
5. **test_no_contradiction_kb:**
 - Tests a KB with no contradictions.
 - Expected result: `False`.

5. Features/Bugs/Missing

Exploring the intricacies: Features enhancing, bugs disrupting, and gaps awaiting attention in our product's functionality.

Figure 1: Test Failures

```

FAILED (failures=3)
(base) idrisanver@Idriss-MacBook-Air-2 InterferenceEngine % python Testcases.py
.....F..F.....
=====
FAIL: test_chain_of_implications (__main__.TestForwardChaining.test_chain_of_implications)
=====
Traceback (most recent call last):
  File "/Users/idrisanver/Dropbox/Mac/Documents/InterferenceEngine/Testcases.py", line 139, in test_chain_of_implications
    self.assertEqual(result, 'YES: a, b, c, d')
AssertionError: 'NO' != 'YES: a, b, c, d'
- NO
+ YES: a, b, c, d

=====
FAIL: test_multiple_implications (__main__.TestForwardChaining.test_multiple_implications)
=====
Traceback (most recent call last):
  File "/Users/idrisanver/Dropbox/Mac/Documents/InterferenceEngine/Testcases.py", line 151, in test_multiple_implications
    self.assertEqual(result, 'YES: a, c')
AssertionError: 'NO' != 'YES: a, c'
- NO
+ YES: a, c

-----
Ran 16 tests in 0.001s

FAILED (failures=2)

```

The ForwardChaining algorithm doesn't have the ability to chain multiple implications of the inference engine, neither does it have the ability to chain implications.

In our recent Python coding project, we did not implement a mechanism to validate user

input. This oversight means that the program currently accepts any data entered by the user, regardless of whether it is in the expected format or within the acceptable range of values. Without input validation, there is a significant risk of encountering errors or unexpected behaviour during runtime, especially if the input is incorrect or malicious. This can lead to crashes, incorrect outputs, or vulnerabilities that could be exploited. To ensure the program runs smoothly and securely, it is crucial to add validation checks that verify the input meets the required criteria before processing it. This step would enhance the program's robustness, user experience, and overall reliability. Moving forward, incorporating comprehensive input validation will be a priority in order to mitigate potential issues and ensure the program functions as intended under various conditions.

Output

```
(base) idrisanver@Idriss-MacBook-Air-2 InterferenceEngine % python InferenceEngine.py TT test.txt
YES: 3
(base) idrisanver@Idriss-MacBook-Air-2 InterferenceEngine % python InferenceEngine.py FC test.txt
YES: a, b, c, d, e, f, p1, p2, p3
(base) idrisanver@Idriss-MacBook-Air-2 InterferenceEngine % python InferenceEngine.py BC test.txt
YES: p2, p3, p1, d
(base) idrisanver@Idriss-MacBook-Air-2 InterferenceEngine % python InferenceEngine.py RB test.txt
input:  p2=>p3 ^ p3=>p1 ^ c=>e ^ b&e=>f ^ f&g=>h ^ p2&p1&p3=>d ^ p1&p3=>c ^ a ^ b ^ p2 d
Knowledge Base: p2=>p3 ^ p3=>p1 ^ c=>e ^ b&e=>f ^ f&g=>h ^ p2&p1&p3=>d ^ p1&p3=>c ^ a ^ b ^ p2
Iteration 0:
  ['p2=>p3 ']
  [' p3=>p1 ']
  [' c=>e ']
  [' b&e=>f ']
  [' f&g=>h ']
  [' p2&p1&p3=>d ']
  [' p1&p3=>c ']
  [' a ']
  [' b ']
  [' p2']
  ['-r']
Result of Resolution: False
```

Figure 2: Output of our program

6. Research (if applicable)

Through rigorous investigation, this research delves into the intricacies of the inference engine, aiming to illuminate its complexities and implications. For this research task we aim to implement a resolution based algorithm. Resolution is a powerful rule of inference used to deduce conclusions from a set of premises in propositional logic. The core idea is to derive a contradiction from the knowledge base combined with the negation of the query. If a contradiction is found, it confirms that the original query logically follows from the knowledge base. Using knowledge from websites and tutorials we have gained a deep understanding on the resolution based algorithms. Below is how we integrated the resolution based algorithm into our program.

The first step is to negate the query and add it to the knowledge base, then transform the

sentences in the knowledge base and the negated query into a standard format called clausal form. Clauses are disjunctions of literals. Finally the resolution process involves iteratively applying the resolution rule to pairs of clauses to derive new clauses until either an empty clause is produced (indicating a contradiction) or no more new clauses can be generated.

And example of an input for this is:

```
sentence = "(PvQ)^(-PvR)^(-QvR)"
```

```
query = "R"
```

By mastering the resolution-based theorem proving algorithm, we will be well-prepared to tackle complex logical reasoning problems, significantly enhancing your understanding and performance in courses related to artificial intelligence and formal logic.

7. Team summary report (if applicable)

In this comprehensive summary, we highlight our team's collective efforts, achievements, and challenges encountered during the development of the inference engine for propositional logic.

Team Members:

- Idris Anver
- Harry Ward

Achievements and Contributions:

- **Idris Anver:**
 - Assisted in solving errors with the Forward Chaining Algorithm
 - Implemented the Truth Table algorithm
 - Implemented and verified 16 test cases to ensure the accuracy and reliability of the algorithms.
- **Harry Ward:**
 - Developed the InferenceEngine.py component.
 - Implemented the Forward Chaining algorithm.
 - Conducted research and developed the ResolutionBased.py component for the inference engine.

Challenges:

- Addressing errors in the Forward Chaining algorithm and ensuring its correctness.
- Ensuring comprehensive testing with all 16 test cases to validate the inference engine's performance.
- Implementing the ResolutionBased was difficult due to capitalisation.

Goals:

- To create a robust and efficient inference engine capable of handling various logical propositions.
- To conduct thorough testing and research to enhance the understanding and application of inference algorithms.

Through collaborative effort and dedicated problem-solving, our team successfully developed a functional inference engine while overcoming several challenges. The project's success underscores the importance of teamwork and the collective contributions of each member.

8. Conclusion

This report described the use of the Truth Table (TT), Forward Chaining (FC), and Backwards Chaining (BC) algorithms to implement an inference engine for propositional logic. Every algorithm has particular advantages. While FC iteratively infers new information from known facts, BC strategically works backwards from a goal to determine necessary conditions, and TT ensures thorough validation by evaluating all possible truth values.

Some limitations were found during testing, especially with the way FC handled multiple implications. Future advancements will depend on these problems being resolved. The research is based on a ResolutionBased Inference engine. Furthermore,

9. Acknowledgements/Resources

In gratitude, we extend our appreciation to those whose contributions and resources enriched this endeavour beyond measure.

10. References

- [1] (example) Google (2019) *Google terms of service*. Available at: <https://policies.google.com/terms?hl=en-US> (Accessed: 27 January 2020).
- [2] youtube(2015) *Resolution to Evaluate Knowledge Bases*. Available at: https://www.youtube.com/watch?v=rFjSH-RA8So&ab_channel=FranciscoIacobelli. (Accessed: 20 May 2024).
- [3] ScienceDirect (2015) *Resolution Principle*. Available at: <https://www.sciencedirect.com/topics/computer-science/resolution-principle#:~:text=The%20resolution%20principle%20yields%20a,a%20list%20of%20true%20axioms>. (Accessed: 20 May 2024).
- [4] U. Schoning, R. Pruim, “Gems of Theoretical Computer Science”, Chapter 6 Springer, 1998. (Accessed: 20 May 2024).
- [5] Radke, P. (2023) *Forward chaining vs. backward chaining in Artificial Intelligence, Built In*. Available at: <https://builtin.com/artificial-intelligence/forward-chaining-vs-backward-chaining#> (Accessed: 24 May 2024).